

Introduction to the Fir cluster

ALEX RAZOUMOV
alex.razoumov@westdri.ca



SIMON FRASER
UNIVERSITY



**Digital Research
Alliance** of Canada

Intro

- Fir is hosted and operated by Simon Fraser University within Canada's national digital research infrastructure
- Replaces older Cedar cluster
- Fir was open to all users on August 12th
- Named after the Douglas fir, a conifer renowned for its towering height, symbolizing the cluster's computational power
- Compared to Cedar: 3-4X increase in CPU-core compute and memory, GPU number down but power up by 3.5X
 - in my tests, a single core is $\sim 5.5X$ faster than on Cedar
 - ranked #78 on the June 2025 TOP500 list ← using a subset of the cluster during assembly
- Fir cluster is ready to support RAC'2025 allocations and general research workloads
- Some services are not set up yet – more on these later

Alliance's national HPC clusters

- 2025 infrastructure renewal https://docs.alliancecan.ca/wiki/Infrastructure_renewal

- Cedar → Fir
- Béluga → Rorqual
- Graham → Nibi
- Niagara → Trillium
- Narval stays the same for now

- Total: 5 175 nodes, 776 912 cores, 2 152 GPUs

- Area size $\propto$ max theoretical single-precision FLOPS

- plot shows overall max theoretical compute capability
- in reality not so GPU-heavy $\Leftarrow$ less than 100% GPU efficiency for complex problems + problem-dependent
- not accurate for older CPUs/GPUs
- plot does not include AI / other specialized clusters

- Partitions coloured by GPUs/node

- One Alliance account to access all systems

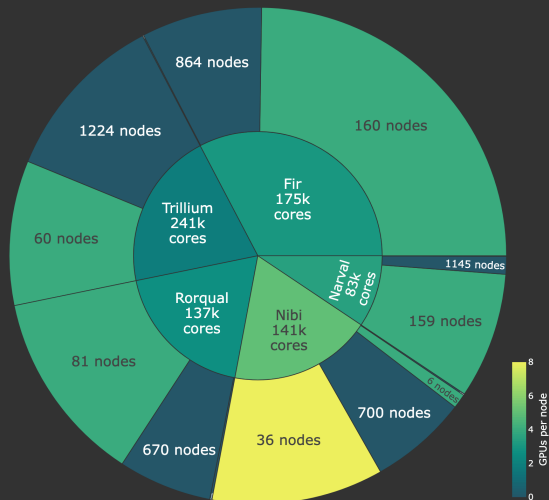
- Fir, Nibi, Rorqual, Narval are general-purpose clusters

- Trillium is for large parallel jobs: scheduling by node only, in multiples of 192 cores

- Most GPUs on the new clusters are NVIDIA's H100s

- Mostly identical software environment (via modules)

- Job sizes, max runtimes, filesystem specifications, quotas, and policies vary across clusters



- Interactive version of this plot

<https://wgpapes.netlify.app/files/clusters.html>

Access to Fir

- Similarly to the other national systems, users must request access via https://ccdb.alliancecan.ca/me/access_systems

- i.e. log in to CCDB and then select Resources | Access Systems
- if you have used Cedar recently, we have preemptively granted you access to Fir
- can always verify your access at the address above
- for certain systems, might need to accept agreements on the same page
- can take up to 1hr for your access to be enabled

- Like you did for Cedar, connect via ssh to `fir.alliancecan.ca`
- 3 login nodes at the moment, randomly land on one of them
 - limits on user processes: similar to Cedar

Access Systems

To use our services you must request access to each individually and accept any corresponding agreement(s), when applicable. If at some point you no longer need access, you can also indicate so in this page.

HPCCloudArtificial Intelligence

Béluga	☒
Fir	☒
HPSS	☒
Nerval	☒
Nibi	☒
Rorqual	☒
Trillium	☒

Alexei Razoumov was granted access on 2025-08-12

Fir Access Request

Access to Fir may be contingent on the acceptance of additional policies. If applicable, these policies will be displayed below and must be reviewed and accepted individually before access is granted.

☒ I request access

☐ I no longer need access

System highlights

Details at <https://docs.alliancecan.ca/wiki/Fir>

- 175,104 CPU cores
- **864 base nodes**: 192 cores, 750 GB memory, 2x AMD EPYC 9655 (Zen 5) processors @2.7 GHz
 - familiar ratio ~4GB/core for base nodes
 - best practices later in scheduling
- **8 large-memory nodes**: 192 cores, 6000 GB memory, 2x AMD EPYC 9654 (Zen 5) processors @2.4 GHz
- **160 GPU nodes**: 48 cores, 1125 GB memory, 1 x AMD EPYC 9454 (Zen 4) processor @2.75 GHz
- 51 PB high-performance DDN Lustre storage with 3 partitions: **/home** , **/scratch** , **/project**
- InfiniBand NDR interconnect
- Operating System: AlmaLinux 9.6
 - familiarity with the Linux command line is essential ⇒ attend our courses (Intro to bash command line + Intro to HPC) throughout the year
- Direct-to-chip liquid cooling
- Full access to the internet from the compute nodes

Filesystems

Details in the table at <https://docs.alliancecan.ca/wiki/Fir#Storage>

- 51 PB high-performance DDN Lustre storage: no spinning drives now, all SSD-based
 - Shared by 3 mounts (`/home` , `/scratch` , `/project`), with different quotas and policies
 - No automatic compression and deduplication
 - built-in compression on physically separate `/nearline` ← separate slide
 - Where to find `/scratch` and `/project`) directories
 - `$SCRATCH` and `~/scratch` both point to `/scratch/$USER`
 - `$PROJECT` points to `~/project` that links to your default project `/project/<gid>`
 - `~/projects` contains links to all your projects `/project/{<gid1>, <gid2>, ...}`
 - `/scratch` purge policy: same as on Cedar, following the Alliance's procedure
 - ~6 weeks before purging files, several email warnings
 - Every group has a `/project` directory with a 1 TB quota
 - `/project` filesystem uses the new per-project quota, rather than file's GID
- ⇒ all files inside of the project apply against the PI's quota independently of the UID/GID
- ⇒ this eliminates issues often faced with tools like `git` , `mv` , or `tar`
- ideally, request an increase via RAC
 - can temporarily increase your regular (non-RAC) allocation

Quotas and data transfer

- Command `quota [--home] [--scratch] [--project] [--nearline]`
 - shorter alias for `diskusage_report`
- Cedar → Fir data migration already happened
 - no action required, as the two clusters shared the same mount points for a while
- For `rsync` / `scp` transfer, use the login node ← no dedicated data transfer node(s) for now
- Globus endpoint: `alliancecan#fir-globus`

Local scratch

- Now 7TB SSD on each compute node
- Inside jobs `$SLURM_TMPDIR` points to `/localscratch/${USER}.${SLURM_JOBID}.0`
- Must copy data out before your job ends
- On both login and compute nodes, `/tmp` is a RAM disk for short-term usage
 - any file(s) written to `/tmp` will count toward cgroup's memory usage

Tape-based nearline

- Cedar's Nearline should be automatically available on Fir
 - moved from Cedar as is, no new hardware
 - by default available to PIs only, default quota ~10TB (policy still under discussion)
- Use `tar` or `dar` to create an archive directly in `/nearline`, keep the source files in their original location elsewhere; ideal file size: 10GB - 4TB

```
NEAR=~ /nearline/def-razoumov-ac/razoumov
cd ~/projects/def-razoumov-ac/razoumov/visThis2019-airfoil
dar -s 10G -w -c $NEAR/monday $(for d in processor?; do echo "-g_$d"; done) # two slices: 10G and 8.2G
dar -l $NEAR/monday > $NEAR/monday.index # create an index file; can be stored in /nearline
```

- No need to compress your data (built-in hardware compression), Ok with already compressed files
- Only available on login and data-transfer (preferred, coming later) nodes, not on compute nodes
- After ~1d, the system will copy the file(s) to tape

```
quota --nearline --per_user --all_users # check the Location field, e.g. ``On disk and tape``
```

- Later the disk copy may be deleted, although the file(s) will still appear in the the directory listing

```
lfs hsm_state $NEAR/monday.?.dar # to check file's location, e.g. ``exists archived``
```

- To mark a file for restoring, either run `lfs hsm_restore <file>` or simply try to access it
- Might take over 1h to retrieve from tape
- Tape copy of deleted files will be retained for up to 60d, but will need the [file's full path to restore](#)

Networking

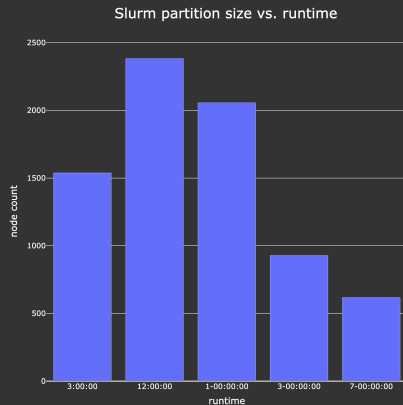
- InfiniBand NDR (Next Data Rate) interconnect
- Base 864 nodes grouped into 4 islands
- Between the islands:
 - 27:5 blocking factor, i.e. will saturate at 5/27 of max theoretical speed when using network simultaneously from all nodes
- Inside each island
 - at least 100 Gb/s and symmetric for all nodes in pairs, ~ 90 ns port-to-port latency
 - 216 nodes per island organized into 108 trays (2 nodes per tray)
 - single 200 Gb/s to the first node in each tray with another followup cable to the second node
- GPU nodes utilize 2:1 blocking factor
- Storage access is fully non-blocking

Software

- Mostly the same as on Cedar
- StdEnv/2023 is now default (StdEnv/2020 still there but hidden)
- cuda/12.6 is default on all new national clusters
- Recompile your code
- Redo the scalability test of your CPU parallel code
- Reinstall your Python virtual environments
- In my case, on first login it was complaining about loading an old intel/2020.1.217
⇒ had to run:
`module reset # restore modules from your "default" or system's default`

Scheduler

- No new Slurm features compared to Cedar
- Jobs runtimes now range from 1hr to 7d
 - >7d jobs no longer allowed across the Federation
 - highly unlikely that we'll enable 28d jobs like in the past
- No CPU overcommit in Slurm (now or in the plans)



- processing output of `sinfo`
- 43 Slurm partitions at the moment, grouped here into 5 “runtime bins”
- many nodes can accept jobs from multiple partitions, so the total number of nodes in the plot exceeds the physical number of nodes

864 base CPU nodes: memory architecture and scheduling

- Each node: 192 cores, 750 GB memory, 2x AMD EPYC 9655 (Zen 5) processors @2.7 GHz
- Two sockets with 96 CPU cores each, organized into 12 chiplets
- Each chiplet is a separate NUMA (non-uniform memory access) node with 8 CPU cores
- Memory attached to a socket $\Rightarrow$ all chiplets/cores within the same socket have uniform local memory access (subject to local bandwidth limitations), slower remote memory access to the other socket
 - in other words, memory latency/bandwidth depend on core and memory locality
- `#SBATCH --cpus-per-task=8` will ensure that all threads in a task stay within a single NUMA node $\leftarrow$ ideal for memory-bound tasks
- `#SBATCH --ntasks-per-node=24` will launch 24 tasks per node, to fully utilize all chiplets on a node when combined with the previous flag (assuming your code can scale to 192 cores)

160 GPU nodes: memory architecture and scheduling

- Each node: 48 cores, 1125 GB memory, 1 x AMD EPYC 9454 (Zen 4) processor @2.75 GHz
- One socket with 48 cores, organized into 6 chiplets and 4 NUMA nodes, i.e. NUMA nodes don't align perfectly with chiplets
 - ⇒ each chiplet is 8 cores, and each NUMA node is 1.5 chiplets (12 CPU cores)
- If your job is memory-bound, you want all your threads in a task to stay within a single NUMA node
 - `#SBATCH --cpus-per-task=12` to bind tasks to NUMA nodes
 - combine with `#SBATCH --ntasks-per-node=4` if need better utilization of the entire node
- `#SBATCH --cpus-per-task=8` might be even better if compatible with your code / workflow
 - will place all threads within one chiplet **and** within one NUMA node
 - final recommendation depends on each code and how much data your send to/from GPU
 - ⇒ study your code's performance with both
- 4 NVidia H100 80GB accelerators per each GPU node ⇒ 640 GPUs total

GPUs: full cards

- Cedar → Fir transition from older P100 / V100 / A100 to fewer modern H100 GPUs
- Four H100 GPUs per node
- Unlike on Cedar, on Fir *must* specify the GPU type
- Use either `--gpus-per-node=<gpuType>:<N>` or `--gpus=<gpuType>:<N>`
- Currently four “GPU types” are supported:
 - `h100` ← full H100-80gb
 - `nvidia_h100_80gb_hbm3_3g.40gb` ← MIG partition
 - `nvidia_h100_80gb_hbm3_2g.20gb` ← MIG partition
 - `nvidia_h100_80gb_hbm3_1g.10gb` ← MIG partition
- For jobs that can utilize full GPUs, use one of these:
 - `--gpus-per-node=h100:<N>` to request N full H100-80gb GPUs per node, where N=1..4
 - `--gpus=h100:<N>` to request N full H100 GPUs spread anywhere on the cluster
 - vast majority of codes won't be able to fully utilize an H100 card ... which is a problem for a ~USD25k card
- Billed at 12.2 RGUs (Reference GPU Units)

Why a job might not utilize an H100 GPU efficiently?

- The computational problem is too small to saturate the GPU
- Bursty GPU usage, e.g. due to many serial parts in the code, or too much compute on the host (CPU-bound) for data pre-processing
- Overhead from too many tiny kernel launches
- The computational problem is inherently memory-bound instead of compute-bound
- Other data transfer issues, e.g.
 - NUMA locality (CPU cores on a different socket than the GPU)
 - NVLink bandwidth
 - slow input data read from network storage
- Using old/unoptimized libraries (lack of optimization for H100)
- Other constraints / bottlenecks resulting in an intermittent GPU utilization pattern

Estimating GPU utilization

- A (GPU) **job monitoring portal** on Fir is coming, but not earlier than the end of September
 - for now can use the portals on Nibi (<https://portal.nibi.sharcnet.ca>) or Rorqual (<https://metrix.rorqual.alliancecan.ca>), as these clusters have exactly the same H100 cards
- User access to **GPU performance counters** on Fir is currently disabled
 - ⇒ GPU profiling by user directly on the system is not available
 - multi-tenant security reasons: could expose things like memory access patterns, instruction throughput
 - same on Nibi
 - on Narval and Rorqual, GPU profiling is possible but requires disabling DCGM (NVIDIA's Data Center GPU Manager to collect metrics) during job submission
 - details in https://docs.alliancecan.ca/wiki/Using_GPUs_with_Slurm#Profiling_GPU_tasks
- At the moment, the only option on Fir is to **attach to an active GPU job** to run **nvidia-smi**
 - **nvidia-smi** does not require GPU performance counters to report GPU utilization, takes it directly from the driver's activity monitoring
 - currently shows GPU utilization on full H100 cards, along with the total power usage (out of max 700W)
 - GPU utilization on MIGs should be enabled later
 - bash functions to simplify running this from the login node:

```
function job-nvidia-smi() {  
    srun --jobid=$(squeue -u $USER | awk 'NR==2_{print_$1}') --pty nvidia-smi  
}  
function job-nvidia-smi-continuous() {  
    srun --jobid=$(squeue -u $USER | awk 'NR==2_{print_$1}') --pty watch -n 1 nvidia-smi  
}
```

GPU utilization via `nvidia-smi` on Fir

```

+-----+
| NVIDIA-SMI 580.65.06                Driver Version: 580.65.06          CUDA Version: 13.0          |
+-----+-----+-----+-----+-----+-----+-----+-----+
| GPU  Name                Persistence-M | Bus-Id                Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.               |
+=====+=====+=====+=====+=====+=====+=====+=====+
|   0   NVIDIA H100 80GB HBM3           On | 00000000:4C:00.0 Off |           0          |
| N/A    34C    P0              268W / 700W | 76827MiB / 81559MiB |    100%      Default |
|                                           |                       | Disabled          |
+-----+-----+-----+-----+-----+-----+-----+-----+

+-----+
| Processes:                                     |
|  GPU   GI    CI          PID    Type    Process name                        GPU Memory |
|          ID    ID                                   Usage                        |
+=====+=====+=====+=====+=====+=====+=====+=====+
|     0   N/A   N/A         4078424      C   ...eFactorization/primesGPU_real    76816MiB |
+-----+

```

GPU utilization via the job monitoring portal on Nibi

1. Run your code on Nibi

```
source ~/startSingleLocaleGPU.sh # normally should be
                                # 'module load cuda' + your code
cd ~/chapelCourse/primeFactorization
chpl --fast primesGPU.chpl
bat gpu.sh
```

File: gpu.sh

```
1 | #!/bin/bash
2 | #SBATCH --time=00:60:00
3 | #SBATCH --mem-per-cpu=3600
4 | #SBATCH --gpus-per-node=h100:1 # full H100
5 | #SBATCH --account=def-someuser
6 | ./primesGPU --n=10_000_000_000
```

sbatch gpu.sh

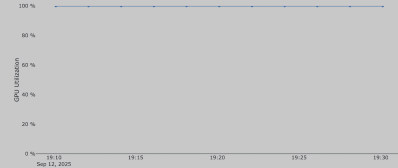
- **caution:** run for at least 10m, as data points are only collected occasionally, and you'll need enough of them for a *reliable graph*

2. Log in to <https://portal.nibi.sharcnet.ca>

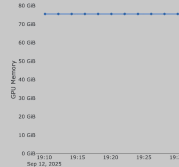
3. Job stats, select your jobID, scroll down to GPU section

GPU Compute cycle used

Graph legend and explanation



GPU Memory used



GPU utilization via Nsight package profiler on Rorqual

NVIDIA's Nsight GUI profiler

1. Generate a profiling report on the GPU

```
source ~/startSingleLocaleGPU.sh    # normally should be 'module load cuda' + your code
cd ~/chapelCourse/primeFactorization
chpl --fast -g --gpu-ptxas-enforce-optimization primesGPU.chpl
DISABLE_DCGM=1 salloc --time=0:60:0 --mem-per-cpu=3600 \
    --gpus-per-node=h100:1 --account=def-someuser
while [ ! -z "$(dcgmi -v | grep 'Hostengine_build_info:')" ]; do sleep 5; done
                                     # wait until DCGM is disabled
nsys profile --trace=cuda -o 111 ./primesGPU -nl 1 --n=10_000_000_000
>>> exit the interactive job after the calculation is done
```

2. Use GUI profiler via <https://jupyterhub.rorqual.alliancecan.ca>

- log in, select JupyterLab interface, no GPU needed
- launches a Slurm job – wait for it to start and then for the JL server to start
- start a desktop, open a terminal, type the commands below

```
module load cuda/12.6
cd ~/chapelCourse/primeFactorization
nsys-ui 111.nsys-rep
```

If your code's GPU utilization is $\lesssim 75\%$, what can you do?

1. scale up your problem to fill the GPU, keeping an eye on the utilization number – might be easier said than done
2. use a MIG partition (next slide)
3. compute on CPU cores

GPUs: static MIG partitions

- If your job uses $< \frac{1}{2}$ compute and $< \frac{1}{2}$ available memory of a full GPU $\Rightarrow$ run it on a MIG partition

~half of all GPUs on Fir are partitioned with MIG (Multi-Instance GPU) technology into 8 smaller virtual GPU instances of which 7 are available to users $\Rightarrow$ replace the GPU flag in your script with one of these:

- `--gpus=nvidia_h100_80gb_hbm3_1g.10gb:1` to request one 1g.10gb partition, at 1.74 RGUs
- `--gpus=nvidia_h100_80gb_hbm3_2g.20gb:1` to request one 2g.20gb partition, at 3.48 RGUs
- `--gpus=nvidia_h100_80gb_hbm3_3g.40gb:1` to request one 3g.40gb partition, at 6.1 RGUs
- Usage billed accordingly, resulting in higher priority compared to allocating a full GPU
- Do not ask for multiple MIG instances in a single job
 1. on the same GPU: not supported in our Slurm config $\Rightarrow$ to run multiple independent tasks, use MPS (next slide) rather than MIG
 2. across multiple GPUs: no support in MIG for data transfer between GPUs over NVLink / NVSwitch
- No graphics APIs (OpenGL, Vulkan) are supported in MIGs
- Our wiki mentions the max number of CPU cores per GPU type: 1 for H100-1g.10gb, 3 for H100-2g.20gb, 6 for H100-3g.40gb, 12 for H100-80gb
 - currently not enforced, but a good practice to follow, otherwise you'll be billed for higher usage
 - may be enforced in the future
- MIG details at https://docs.alliancecan.ca/wiki/Multi-Instance_GPU

GPUs: dynamic sharing via MPS (Multi-Process Service)

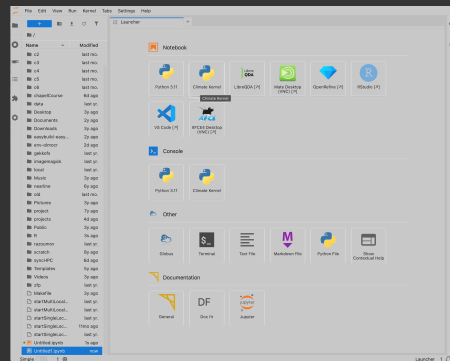
- Alternative technology: dynamic sharing of a GPU (cores + memory) between unrelated processes
 - all processes must belong to the *same user*: a GPU can't be safely shared with MPS between different users
- No need to rewrite or even recompile your code!
- Any number of MPS clients (processes) between 2 and 48 (for A100 and H100 GPUs)
 - much higher granularity than MIG
 - e.g. a group of MPI ranks sharing a single GPU
 - or multiple serial GPU codes (GPU farming)
- Launch the MPS daemon from inside your Slurm job submission script
 - all user processes (including the daemon) are killed automatically at the end of the job
- MPS can be combined with MIGs
- Details at https://docs.alliancecan.ca/wiki/Hyper-Q/_/MPS

Visualization

- H100 GPUs do not support graphics rendering efficiently $\Rightarrow$ don't use them for visualization
 - OpenGL / Vulkan pipelines can run only on 2/66 thread controllers on an H100 $\Rightarrow$ rendering will utilize the card only at $\sim 3\%$ efficiency ... not the best use of this expensive device
- Instead, we recommend using CPU nodes for visualization in the vast majority of cases
 - much faster to load *partitioned* data in parallel onto 16/32/64/128 CPU cores $\Rightarrow$ why not use the same cores for parallel rendering?
 - far more CPU cores than GPUs $\Rightarrow$ much easier to allocate for batch rendering
 - modern libraries can do both rasterization and ray tracing on CPU cores
 - we teach remote parallel visualization (interactive + batch off-screen) in our courses
- Nibi has AMD MI300A nodes with unified CPU+GPU memory – to be tested for vis
- SHARCNET is planning to make some older T4's available for rendering (interactive jobs or Open OnDemand) on Nibi

Interactive access

- ~ 10 nodes in the interactive partitions:
 - `sinfo | grep interac`
 - 3hr max runtime
 - use `salloc` to start interactive jobs
- <https://jupyterhub.fir.alliancecan.ca> now online
 - log in, select JupyterLab interface and other parameters
 - launches a Slurm job – wait for it to start and then for the JL server to start
- Open OnDemand (OOD) is planned, but no firm details yet on what services it'll offer
 - will use a number of dedicated nodes outside of Slurm
 - might or might not add a Slurm partition to it later
 - lower priority; end of October?
- Can also use graphical interfaces on other clusters:
 - <https://jupyterhub.rorqual.alliancecan.ca> will start a Slurm job
 - <https://ondemand.sharcnet.ca> on Nibi
 - <https://ondemand.scinet.utoronto.ca/pun/sys/dashboard> on Trillium



Fir Cloud

- The team is working on it, planning to make it available towards the end of September
 - there will be a separate email update from the Alliance when it is ready
 - watch https://docs.alliancecan.ca/wiki/Cloud#Cloud_systems
- For now RAC'2025 cloud allocations remain on Cedar Cloud
<https://cedar.cloud.computecanada.ca>, until Fir Cloud is open
- Will provide RAC + default allocations (same as Cedar cloud)
 - default allocations will not be enabled by default ⇒ submit a ticket support@tech.alliancecan.ca
- Fir Cloud and Arbutus should be very similar, more so than Cedar and Arbutus

Summary

- Fir is ready for production, both in general queues and RAC allocations
- Minimal changes in software and scheduling, use Fir as you would use Cedar
- May need to recompile your code
- Pay attention to parallel scalability of your CPU code
- Biggest changes: GPU scheduling and utilization, MIG partitions
- Some features still in the pipeline: job monitoring portal, Fir Cloud, Open OnDemand, dedicated data transfer node(s)
- Cedar should be fully decommissioned as of Sep-12
 - with the exception of Cedar Cloud for now
 - part of the old cluster will be repurposed as an internal SFU resource for researchers and students
- Watch Sergey Mashchenko's (SHARCNET) July 2025 webinar *Migrating to the upgraded national systems*
- Any problems ⇒ email support@tech.alliancecan.ca to open a ticket
 - as usual, provide as much detail as you can: specific error messages, jobID, Slurm script, software version, etc.

Questions?